

Documentación Técnica y Manual de Procesos: Sistema de Reportes Farmapatria

1. Introducción y Propósito

El **Sistema de Reportes Farmapatria** es una plataforma web integral diseñada para la gestión, seguimiento y resolución de incidentes o requerimientos técnicos (tickets) dentro de la red de farmacias y gerencias de Farmapatria.

El propósito de esta documentación es detallar la arquitectura del sistema, su configuración, la estructura de la base de datos y la lógica de negocio subyacente. Este documento sirve como la base técnica fundamental para el **Manual de Procesos y Procedimientos** de la organización, permitiendo a los desarrolladores, administradores y personal técnico entender cómo interactúan los componentes del sistema.

2. Arquitectura Tecnológica (Stack)

El sistema está construido bajo una arquitectura Cliente-Servidor separada, utilizando tecnologías modernas de desarrollo web:

2.1. Frontend (Cliente)

- **Framework:** Angular 21 (TypeScript).
- **Arquitectura Estructural:** Código desacoplado utilizando una Capa de Servicios (`AuthService` , `TicketService` , `AdminService`) para la comunicación HTTP, con Modelos de Dominio estrictos (`Interfaces`) para garantizar tipado seguro.
- **Estilizado:** Tailwind CSS v4 (Enfoque utilitario y diseño responsivo).

- **Gráficos y Visualización:** Chart.js (Para KPIs y reportes estadísticos en el dashboard de administrador).
- **Generación de Documentos:** jsPDF (Para exportación de reportes).
- **Manejo de Rutas:** Angular Router con Guards de protección (AuthGuard, RoleGuard).
- **Renderizado:** Soporte para Server-Side Rendering (SSR) incluido en la configuración.

2.2. Backend (Servidor)

- **Entorno de Ejecución:** Node.js.
- **Framework:** Express.js v5.
- **Arquitectura Estructural:** Patrón MVC (Modelo-Vista-Controlador). El código está modularizado en `routes/` (enrutamiento), `controllers/` (lógica de negocio) y `middlewares/` (seguridad), eliminando el anti-patrón de backend monolítico.
- **Seguridad y Autenticación:**
 - `jsonwebtoken` (JWT) para control de sesiones.
 - `bcryptjs` para el hash y encriptación de contraseñas.
- **Manejo de Archivos:** `multer` para la subida de adjuntos (Imágenes y PDFs, hasta 5MB).
- **Conexión a Base de Datos:** `pg` (PostgreSQL Client).
- **Configuración:** `dotenv` para gestión segura de variables de entorno.

2.3. Base de Datos

- **Motor:** PostgreSQL.
- **Estructura Relacional:** Tablas interconectadas mediante claves foráneas (Usuarios, Tickets, Notificaciones).

3. Modelo de Base de Datos y Entidades

El sistema depende de tres entidades principales:

3.1. Tabla: `usuarios`

Gestiona la identidad y el acceso al sistema.

- **Campos Principales:**

- `id` (PK, Serial)
- `cedula` , `email` (Únicos)
- `nombre` , `apellido` , `fecha_nacimiento`
- `password` (Hash BCrypt)
- `gerencia` , `farmacia` (Ubicación del usuario)
- `rol_id` (Identificador de permisos: Ej. 2 para Usuario Estándar/Jefe de Farmacia, 6 para Gerentes Estadales, etc.)
- `estado` (Activo, Inactivo, Bloqueado)
- `avatar` (Ruta al archivo de imagen de perfil)

3.2. Tabla: `tickets`

Almacena los requerimientos e incidentes reportados.

- **Campos Principales:**

- `id` (PK, Serial)
- `numero_reporte` (Formato correlativo: `REP-XXXX`)
- `usuario_id` (FK -> `usuarios.id`)
- `tecnico_id` (FK -> `usuarios.id` - Técnico asignado)
- `estado_ticket` (Pendiente, En Progreso, Sin Confirmar, Resuelto)
- `nivel_reporte` , `tipificacion_falla` (Categorización)
- `unidad_reporta` , `unidad_afectada`
- `anydesk` (Código de acceso remoto)
- `descripcion`
- `archivo_adjunto` (Ruta al archivo subido)
- `fecha_creacion`

3.3. Tabla: `notificaciones`

Sistema de alertas integradas en la plataforma.

- **Campos Principales:**

- `id` (PK, Serial)
- `usuario_id` (FK -> `usuarios.id` - Destinatario de la alerta)
- `mensaje` (Texto descriptivo del evento)
- `leida` (Boolean)
- `fecha_creacion`

4. Módulos y Flujos de Procesos

A continuación se detalla la lógica de negocio de los módulos principales que componen el sistema.

4.1. Módulo de Autenticación y Acceso

1. Registro (/registro):

- El usuario proporciona sus datos personales, laborales y credenciales.
- El sistema encripta la contraseña y autodesigna el `rol_id` basado en la selección de gerencia (ej. si contiene "ESTADAL", asigna rol 6).
- El estado inicial es "Activo".

2. Login (/login):

- Verificación de credenciales (`email` y `password`).
- El sistema bloquea el acceso si el usuario tiene estado `Inactivo` o `Bloqueado` .
- Retorna un token JWT con una validez de 8 horas.

3. Recuperación de Contraseña (/recuperar-contrasena):

- Requiere validación doble cruzada: `email` y `cedula` .
- Permite el establecimiento de una nueva contraseña encriptada en la base de datos sin necesidad de intervención del administrador.

4.2. Módulo de Gestión de Tickets (Ciclo de Vida)

El núcleo del proceso de atención técnica sigue un flujo de estados estricto:

1. Creación (Estado: `Pendiente`):

- El usuario completa el formulario en `/generar-reporte` .
- Puede adjuntar evidencia (PNG, JPG, PDF).
- Se genera un identificador único (Ej. `REP-0042`).

2. Atención (Estado: `En Progreso`):

- Un usuario con privilegios (Administrador/Técnico) visualiza el ticket en su dashboard.
- Hace clic en "Tomar Ticket". El sistema registra el `tecnico_id` y cambia el estado.
- Se notifica automáticamente al creador original.

3. Resolución Técnica (Estado: `Sin Confirmar`):

- El técnico marca el ticket como solucionado desde su panel.

- El estado cambia a `Sin Confirmar`. Se requiere que el usuario original valide que el problema realmente se solucionó.

4. Confirmación de Usuario (Estado: `Resuelto` o Retorno a `En Progreso`):

- El usuario recibe una notificación.
- **Si el problema se solucionó:** El usuario hace clic en confirmar. El estado cambia a `Resuelto` (Cierre definitivo).
- **Si el problema persiste:** El usuario reporta "Error Persistente". El ticket regresa a estado `En Progreso` y se notifica al técnico asignado para que retome el caso.

4.3. Módulo de Administración de Usuarios

Accesible únicamente para perfiles administrativos (Soporte Técnico/Super Admin).

- **Listado Global:** Visualización de todos los usuarios registrados en el sistema.
- **Gestión de Accesos:** Modificación dinámica del `estado` (para inhabilitar el ingreso al sistema) y del `rol_id` (para elevar o restringir privilegios).

4.4. Módulo de Perfil de Usuario

- **Actualización de Datos:** Los usuarios pueden corregir su nombre, apellido y fecha de nacimiento.
- **Gestión de Avatar:** Sistema de carga de imagen de perfil mediante `multer`, almacenada en la carpeta `/uploads` del servidor backend.

5. Descripción de la API (Backend RESTful)

Todas las rutas privadas requieren que se envíe el token JWT en el encabezado:

`Authorization: Bearer <token>`.

Rutas Públicas (Sin Token)

- `POST /api/login` : Iniciar sesión y obtener JWT.
- `POST /api/registro` : Registrar nuevo usuario.
- `PUT /api/recuperar-contrasena` : Restablecer clave con validación de CI.

Rutas Privadas de Usuario (Requieren Token)

- `POST /api/tickets` : Crear un nuevo ticket (Soporta `multipart/form-data`).
- `PUT /api/tickets/:id` : Editar un ticket (Solo si está en estado `Pendiente`).
- `GET /api/tickets/:usuarioId` : Obtener el historial de tickets del usuario logueado.
- `PUT /api/tickets/:id/confirmar` : Confirmar la resolución de un ticket.
- `PUT /api/tickets/:id/error-persistente` : Rechazar resolución y devolver a progreso.
- `GET /api/notificaciones/:usuarioId` : Obtener últimas 10 notificaciones.
- `PUT /api/notificaciones/marcar-leidas/:usuarioId` : Marcar notificaciones como leídas.
- `PUT /api/usuarios/:id` : Actualizar perfil básico.
- `POST /api/usuarios/:id/avatar` : Subir/Actualizar foto de perfil.

Rutas Privadas de Administración (Requieren Token y Privilegios)

- `GET /api/admin/usuarios` : Obtener lista completa de usuarios.
- `PUT /api/admin/usuarios/:id/rol` : Cambiar el rol de un usuario.
- `PUT /api/admin/usuarios/:id/estado` : Cambiar estado (Bloquear/Activar).
- `GET /api/admin/tickets` : Obtener todos los tickets a nivel global.
- `PUT /api/admin/tickets/:id/tomar` : Asignarse un ticket como técnico.
- `PUT /api/admin/tickets/:id/resolver` : Marcar un ticket como resuelto por parte del técnico.

6. Procedimientos de Seguridad y Estabilidad Implementados

1. **Protección de Contraseñas:** Ninguna contraseña se almacena en texto plano. Se utiliza `bcryptjs` con un salt rounds de 10.
2. **Protección de Rutas (Guards en Angular):** El frontend evita que usuarios no autenticados accedan al panel mediante `AuthGuard` . Adicionalmente, `RoleGuard` restringe el acceso a `/panel-admin` basándose en el rol del JWT.
3. **Seguridad de API (Interceptor y Middlewares):** Todas las peticiones HTTP desde Angular inyectan automáticamente el token JWT mediante un Interceptor global (`authInterceptor`). En el backend, las rutas sensibles ejecutan los middlewares `verificarToken` y `esSuperAdmin` para validar firmas y privilegios elevados.

4. **Validación de Archivos:** La carga de imágenes y documentos se filtra estrictamente en el backend utilizando `multer`, limitando por tipo MIME (JPG, PNG, GIF, PDF) y tamaño máximo de 5MB, mitigando riesgos de subida de scripts maliciosos.
 5. **Aislamiento de Variables de Entorno:** Credenciales de base de datos y llaves secretas se mantienen en el archivo `.env`, el cual no se sube al control de versiones.
 6. **Resiliencia y Anti-Crash (Fail-Safe):** Se incorporó un interceptor global de errores (`errorHandler.middleware.js`) en el backend. Este mecanismo atrapa excepciones no controladas a nivel de servidor, previniendo caídas totales ("crashes") y asegurando que el cliente reciba respuestas HTTP estandarizadas en lugar de filtrar información de la base de datos.
-

7. Directrices para el Mantenimiento

- **Levantamiento Local:**
 - Backend: `npm start` o `node server.js` (Puerto 3000).
 - Frontend: `ng serve` (Puerto estándar 4200).
- **Base de Datos:** Asegurarse de que el servicio de PostgreSQL esté en ejecución y que los parámetros del `.env` coincidan con el entorno local o de producción.
- **Almacenamiento de Archivos:** La carpeta `backend/uploads/` debe tener permisos de escritura. En un entorno serverless (como Netlify/Vercel) para producción, esta estrategia de `multer` local deberá migrarse a un servicio Cloud Storage (como AWS S3 o Supabase Storage) ya que el sistema de archivos es efímero.